

Adaptive tree-like neural network: Overcoming catastrophic forgetting to classify streaming data with concept drifts

YiMin Wen^a, Xiang Liu^a, Hang Yu^{b,*}

^a Guangxi Key Laboratory of Image and Graphic Intelligent Processing, Guilin University of Electronic Technology, Guilin, 541004, Guangxi, China

^b School of Computer Engineering and Science, Shanghai University, Shanghai, 200444, China

ARTICLE INFO

Keywords:

Deep neural networks
Streaming data
Concept drift

ABSTRACT

With the development of deep neural networks (DNNs), classifying streaming data with concept drifts based on DNNs is becoming more and more effective. However, the continuous and infinite characteristic of streaming data makes it difficult to set an appropriate depth for a DNN-based model in advance. Moreover, how to improve the model's adaptability to concept drifts while overcoming catastrophic forgetting still remains a difficult issue. To address these issues, an Adaptive Tree-like Neural Network (ATNN) is proposed in this paper. ATNN adaptively increases the depth of its active branch according to the weight of the deepest node in the active branch. Once a new concept is detected, it chooses a suitable position on its trunk to grow a branch for the new concept according to the relation between the Fisher information and the gradient of parameters. Experiments demonstrate the rationality of ATNN to adaptively increase the depth of its branch or to choose a suitable position to grow a new branch, and illustrate that ATNN can more quickly adapt to concept drifts, and continuously improve classification performance for recurring concepts. The code of the proposed algorithm is available at <https://github.com/mlmmwym/ATNN>.

1. Introduction

In many real applications, such as weather prediction, call record analysis, network intrusion detection, and online shopping, a huge amount of data is generated in a streaming way. However, due to working conditions or environment changing over time, the newly arrived data may exhibit a different distribution from the previous ones, referred to as concept drift [1]. For instance, to avoid gateway detection, network intrusions often exhibit dynamics.

Learning from data streams with concept drifts is crucial in the ever-changing and big data environment to provide reliable data-driven predictions and decision facilities [2,3]. The traditional methods for adapting to concept drift can be divided into two categories: single-model methods and ensemble methods. Typical single model methods include decision tree-based methods [4,5], extreme learning machine-based methods [6], nearest neighbor-based methods [7], and so on. Typical ensemble methods [8–10] combine several component classifiers according to specific rules (such as weighted average, combined voting, etc.) to obtain better performance than single classifiers. However, traditional methods mainly utilize shallow models and cannot adapt to the increasingly complex and diverse data in the era of big data [11,12].

In recent years, deep neural networks (DNNs) have achieved tremendous success in various fields of artificial intelligence. Compared with traditional shallow models, DNNs have strong data representation capabilities and can learn complex patterns and features. Therefore, employing DNNs to classify data streams with concept drifts is recognized as a promising technology capable of outperforming traditional methods [11]. Existing DNN-based methods for concept drift adaptation can be mainly divided into two categories: parameter updating methods and structure updating methods [13]. The typical parameter updating methods employ a static model structure that adapts to concept drift only by updating the model's parameters [11,12,14,15]. Typical structure updating methods detect concept drifts and adapt to them by adjusting the model's depth and width. [16,17].

However, two major challenges remain unresolved in existing DNN-based methods. Firstly, the existing methods cannot solve the problem of how to preset an appropriate depth for a DNN-based model. If its depth is set too shallow, its learning capacity will be limited, leading to difficulty in learning complex patterns. On the contrary, if its depth is set over large, its learning process will converge slowly, leading to poor classification accuracy. Although structure updating methods can increase model's depth during training process, they may experience excessive depth growing on some datasets containing frequent concept

* Corresponding author.

E-mail addresses: ymwen@guet.edu.cn (Y. Wen), 21032303090@mails.guet.edu.cn (X. Liu), yuhang@shu.edu.cn (H. Yu).

<https://doi.org/10.1016/j.knosys.2024.111636>

Received 14 November 2023; Received in revised form 22 February 2024; Accepted 12 March 2024

Available online 16 March 2024

0950-7051/© 2024 Elsevier B.V. All rights reserved.

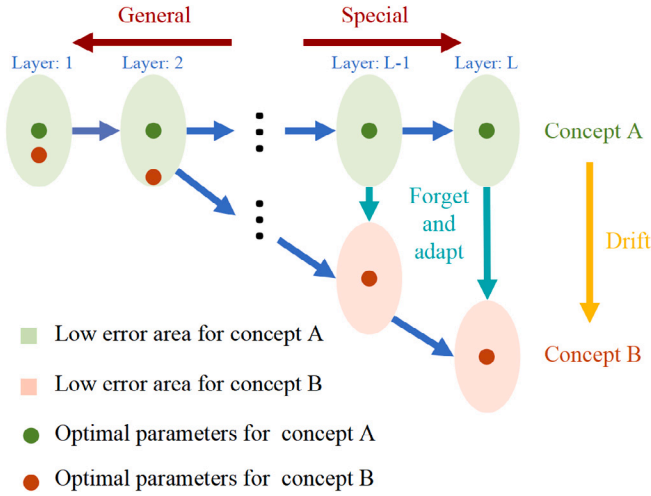


Fig. 1. The forgetting process of parameters in each layer of a DNN model when concept drift occurs. If the parameters leave the low error area of the concept A, it will lead to a significant decrease in the classification performance of the model on the concept A, indicating that the model has forgotten the knowledge of the concept A.

drifts. Secondly, the existing methods try to overcome catastrophic forgetting the just learned when adapting to concept drift. Nevertheless, their new learning is followed by almost complete forgetting of what was previously learned [13], due to the deeper layers of a trained DNN model containing more concept-specific knowledge [18], the DNN-based model with excessive depth are more susceptible to catastrophic forgetting when encountering concept drifts. Fig. 1 illustrates the forgetting process of a DNN model when concept drift occurs.

Multi-branch structure can be considered as an effective approach to address catastrophic forgetting [19]. It is generally applied by sharing hidden layers while keeping branches to learn task-specific knowledge. However, when facing the problem of concept drift adaption, the multi-branch structure model will encounter new challenges: when should the model add a new branch and when should an old branch be retrained? Another challenge is how to choose a suitable position to grow a new branch, which determines the number of parameters shared. If too few parameters are shared, it is difficult to utilize existing knowledge to help the model learn new concepts. On the contrary, if there are too many parameters shared, the negative transfer will occur due to excessive concept-specific knowledge and hinder learning new concept.

To overcome the above challenges of deploying DNN-based methods to classify data streams with concept drifts, an adaptive tree-like neural network (ATNN) is proposed in this paper. Similar to HBP [12], each node in an ATNN model contains a hidden layer and an output layer. The distinctions between ATNN and existing DNN-based methods for data stream classification include: (1) ATNN is a special multi-branch model; (2) During learning process, ATNN can adaptively increase the depth of its active branch or generate a new branch connected to a suitable position in the model for a new concept and activates it; (3) ATNN's classification approach is performed by weighted ensembling the output layer of each node in the active branch. The main contributions of this paper are as follows:

- A new ATNN model is proposed to learn from data streams with concept drifts and meanwhile overcome catastrophic forgetting involved. The proposed model takes concept drift detection to determine when to grow a new branch. The appropriate position for growing a new branch is selected by calculating the hidden layer with the lowest similarity between the Fisher information and the gradient of parameters after concept drift occurs.

- A method is proposed to adaptively increase the depth of an ATNN model when streaming data is stable. If the weight of the deepest node in the active branch becomes the largest, a new node is added behind the deepest one. Each branch adaptively grows to a depth suitable for the learning task at hand without the need to manually preset its depth. The method proposed above can make a branch continue to learn from recurring concept and improve its classification accuracy.
- ATNN is extensively evaluated on several streaming datasets. The experimental results illustrate that ATNN significantly outperforms several state-of-the-art methods.

The rest of this paper is organized as follows. Section 2 is an overview of related work. Section 3 introduces ATNN in detail. Section 4 presents experiments. Conclusions are presented in Section 5.

2. Related work

2.1. Traditional methods for concept drift adaptation

Concept drift means unforeseeable changes in the underlying distribution of streaming data over time. If a trained model cannot adapt to concept drift in time, it will result in poor classification performance [3]. Concept drifts can be distinguished into four types: sudden drift, gradual drift, incremental drift, and recurring concept [3,20]. Recurring concept means previously active concept reappears after some time. It differs from the common seasonality notion in a way that it is not certainly periodic and is not clear when the concept might reappear. However, even though recurring concepts are very common in real applications, most of the current models always forget the learned concepts when they become relevant again due to the stability-plasticity dilemma [20,21].

The existing research on concept drift mainly includes concept drift adaptation and detection. The most straightforward way of adapting to concept drift is to retrain a new model with the latest data to replace the obsolete model [3,22–24]. Another way to adapt to concept drift is ensemble methods [8–10,25], which perform well on recurring concept.

In terms of the test statistics they apply, concept drift detection methods can be divided into three categories: error rate-based drift detection, data distribution-based drift detection, and multiple hypothesis test drift detection [3]. One of the most-referenced error rate-based drift detection methods is the Drift Detection Method (DDM) [26]. It is the first method to introduce warning and drift levels for concept drift detection. The warning level is reached if $p + s \geq p_{min} + 2s_{min}$, while the drift level is reached if $p + s \geq p_{min} + 3s_{min}$, where p represents the error rate in the time window, s represents the standard deviation, p_{min} and s_{min} represent the values corresponding to the minimum sum of p and s , respectively. DDM can serve as a framework of concept drift detection, and its test statistics, window update strategy, detection confidence, etc. can be flexibly adjusted according to specific problems and models to improve its detection accuracy and sensitivity. Similar implementations have been adopted and applied in LLDD [27], STEP [28], EDDM [29], HDDM [30], FW-DDM [31], RDDM [32], AOIL [33] and DELM [34].

Data distribution-based drift detection utilizes a distance function/metric to quantify the dissimilarity between the distribution of historical data and the new data to detect concept drift, such as ITA [35], SCD [36], CM [37], etc. Multiple hypothesis test drift detection utilizes multiple hypothesis tests to detect concept drifts, such as JIT [38], IV-Jac [39], HLFR [40], and so on.

2.2. DNN-based methods for concept drift adaption

Yuan et al. [13] divided the existing DNN-based methods for concept drift adaptation into two categories: parameter updating and structure updating methods.

Parameter updating methods employ static model structures and adapt to concept drift by updating model parameters. Sahoo et al. first proposed the HBP algorithm to train deep neural networks in an online setting to address the slow convergence of deep networks [12]. They amended the existing DNN architecture by attaching every hidden layer to an output layer and proposed a novel Hedge Backpropagation method to online update the parameters of an HBP model effectively. Disabato et al. proposed a method to find out the outdated layers due to concept drift and retrain the outdated layers with the data after concept drift [41]. Guo et al. proposed the SEOA algorithm, which focuses on solving the convergence problem of the model to classify streaming data with concept drifts [11]. SEOA replaces the hidden layers with adaptive depth units and selectively integrates the classifiers of each hidden layer according to the fluctuation of streaming data. Yang et al. proposed the IADM algorithm [14], which employs weighted Fisher regularization to improve the stability of hidden neurons, and utilizes attention networks to adjust the weights of classifiers. However, HBP, IADM, SEOA, and the method proposed by Disabato et al. need to preset an overly large depth for the model before learning and then adaptively adjust the effective capacity of the model during learning process. In fact, if the model's depth is set too large, there will be overfitting problems in its deep layers, resulting in poor classification performance, and the gradient propagated back to shallow layers will interfere with the convergence of the shallow layers.

Structure updating methods adapt to concept drifts by adjusting the depth or width of the network. Ashfahani et al. proposed the ADL algorithm that the classification model can be constructed from scratch with the absence of initial network structure via the self-constructing network structure [16]. ADL utilizes a model structure similar to HBP where each hidden layer is connected to an output layer. Every time a concept drift is detected, a new layer is added behind the deepest layer of the model, while streaming data is stable, ADL utilizes the NS method [42] to determine whether the model is overfitting or underfitting to adaptively adjust its width. Pratama et al. proposed the NADINE algorithm [17], which employs a method similar to ADL to adjust the depth and width of the model. NADINE utilizes memory replay to train new hidden layers. Kauschke et al. proposed the NN-Patching algorithm [43], in which the samples in the most recent batch that are classified incorrectly by the basic classifier are employed to train a patch network and meanwhile an error detector is trained to determine whether to use the basic classifier or the patch network to classify the next data batch. However, ADL, NADINE, and NN-Patching cannot identify recurring concept. When encountering a recurring concept, ADL and NADINE need to train a new layer and NN-Patching needs to retrain the patch network to relearn recurring concepts, rather than effectively leveraging the retained knowledge for faster adaptation. ADL and NADINE may experience excessive depth growth when frequent concept drift occurs. In addition, they cannot effectively overcome catastrophic forgetting while at the same time allowing significantly faster adaptation.

3. Adaptive tree-like neural network

An ATNN model is trained online in a prequential test-and-train mode. In every online iteration, when an instance x_t is observed, and the ATNN model makes a prediction for it, and when its ground truth label is revealed, the model's structure and parameters are then updated.

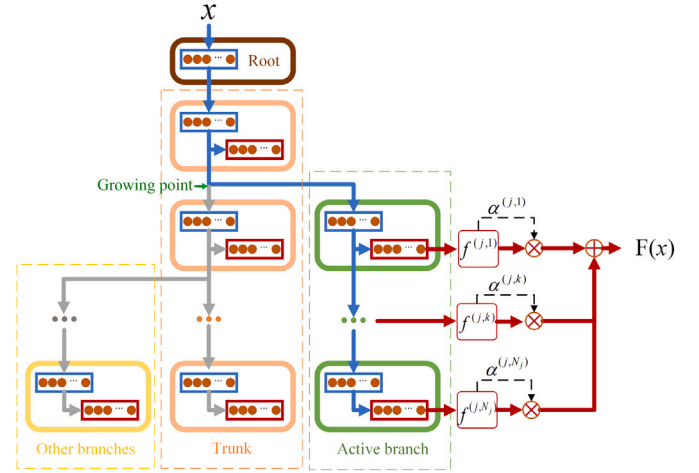


Fig. 2. Blue lines represent feedforward flow for computing hidden layer features. Red lines indicate softmax output at prediction time. The nodes connected by gray lines are not participating in the current classification process.

3.1. Model structure

ATNN is a tree-like neural network (Fig. 2) that contains a root node and multiple concept-specific branches. The root node contains an input layer and is only used for input, while each branch contains multiple nodes. Like HBP [12], each node in an ATNN model contains a hidden layer and an output layer. An ATNN model initially contains a root node and a branch that contains only one node. The number of its branches and the depth of each branch will be adaptively expanded during learning process. The branch learned from the first concept is considered as the trunk. When a new concept is detected, a new branch will be added to the trunk and then activated and named as the active branch. For convenience, the position that the active branch is added to the trunk is called the growing point. When a recurring concept is detected, the branch that corresponds to the recurring concept is activated to learn.

The ATNN method is presented in the algorithm 1.

3.2. Weighted ensemble classification

ATNN utilizes the same weighted ensemble classification method and weight adjustment strategy as HBP [12]. The distinction from HBP is that ATNN has a multi-branch structure, where only one branch is active and only the nodes on the active branch are weighted ensemble for classification. Following is the classification function:

$$F(x_t) = \sum_{k=1}^{N_j^*} \alpha_t^{(j^*,k)} f_t^{(j^*,k)} \quad (1)$$

where t corresponds to the t th training sample, (j, k) corresponds to the k th node on the j th branch. For convenience, j^* is specifically used to refer to the active branch while j_0 is used specifically to refer to the trunk. N_j^* is the total number of nodes on the active branch, f denotes the output of the classifier, and α means classifier's weight. The adjustment strategy for α is as follow:

$$\alpha_{t+1}^{(j^*,k)} \leftarrow \alpha_t^{(j^*,k)} \beta^{E_t^{(j^*,k)}}, \forall k = 1, \dots, N_j^* \quad (2)$$

where β ($0 < \beta < 1$) is a hyperparameter used to adjust the change speed of α . $E_t^{(j^*,k)}$ is the Cross-Entropy Loss of the classification result of the k th node on the active branch for x_t . Similarly, in the following text, $E_t^{(j^*)}$ is used to represent the Cross-Entropy Loss of the active branch for x_t .

A smoothing hyperparameter s ($0 < s < 1$) is employed to adjust the minimum weight for each node. The minimum weight is set to

$\alpha_{min} = s/N_{j^*}$. If the value of α calculated by the formula (2) is less than α_{min} , α will be set to α_{min} . Then, the weights of the nodes on the active branch are normalized as $\sum_{k=1}^{N_{j^*}} \alpha^{(j^*,k)} = 1$.

3.3. How to adaptively increase the depth of the active branch?

Sahoo et al. [12] have observed that when the number of training samples is small, the shallowest layer of an HBP model has the highest value of α . As training samples continue to increase, the layer whose classifier has the highest value of α will be located progressively deeper, and the shallow and the very deep classifiers receive little weight in the last training segment showing HBP's ability to perform model selection. According to the update rule of the classifier's weight of HBP, the highest value of α signifies that the corresponding classifier delivers the best classification performance. Thus, it can be further reasoned from this observation that the location of the highest α value can signal the approximate optimal match between the depth of an HBP model and the size of its training samples, presetting an excessively deep depth can lead to overfitting and reduce generalization performance of an HBP model [16].

In light of the above analysis, we propose a strategy to enable the active branch in an ATNN model to adaptively grow to an appropriate depth. ATNN model adaptively increases the depth of its active branch by adding a new node behind the deepest node $l^{(j^*,N_{j^*})}$ when $\alpha^{(j^*,N_{j^*})}$ becomes the largest. If the active branch only contains one node, it is considered that $\alpha^{(j^*,1)}$ is the largest and a new node is added directly behind it. When a new node is added on the active branch, $N_{j^*} = N_{j^*} + 1$, $\alpha^{(j^*,N_{j^*})}$ is set to $1/N_{j^*}$. Classifier weights of the nodes on the active branch are normalized again.

To explain the rationality of the strategy proposed above, assuming there is an existence of an optimal depth for a branch, denoted as γ . In the early stage of learning, the depth of the active branch is small ($N_{j^*} < \gamma$), resulting in its shallower layers experiencing more significant underfitting. In this case, the node of $l^{(j^*,N_{j^*})}$ exhibits the strongest learning capability. As it converges, $\alpha^{(j^*,N_{j^*})}$ increases, leading to an expansion of the active branch. This iterative process continues, causing the depth of the active branch to gradually increase. When $N_{j^*} > \gamma$, the classification performance of $l^{(j^*,N_{j^*})}$ cannot exceed that of the γ -th layer due to overfitting, so $\alpha^{(j^*,N_{j^*})}$ cannot become the largest, and the depth of the active branch ceases to expand. As a result, the depth of the active branch is adaptively maintained around γ .

3.4. How to overcome catastrophic forgetting?

To overcome catastrophic forgetting, ATNN involves two approaches. Firstly, ATNN employs Weighted Fisher Regularization proposed by Kirkpatrick et al. [44] to constrain the update of the parameters of the trunk's hidden layer to ensure its stability and maintain the relevant knowledge. Secondly, ATNN utilizes concept drift detection technique to determine when to grow a new branch and when to select an old branch to train, ensuring that each branch is trained only on its corresponding concept, preventing concept-specific knowledge from being covered by other concepts. The appropriate position for growing a new branch is selected by calculating the hidden layer with the lowest cosine similarity between the Fisher information and the gradient of the parameters after concept drift, to effectively utilize the general knowledge of the trunk to help the new branch learn new concept.

3.4.1. Weighted Fisher regularization

During online learning process, the loss function proposed by Kirkpatrick et al. [44] is employed to train the active branch and the part of the trunk.

$$L_t = E_t^{(j^*)} + \frac{\lambda}{2} \sum_{k=1}^{N_g} \left\| F^{(j_0,k)} \odot \left(\theta_t^{(j_0,k)} - \theta^{*(j_0,k)} \right) \right\|^2 \quad (3)$$

As in the formula of (3), the loss L_t for the sample x_t includes two parts. The first part $E_t^{(j^*)}$ is the Cross-Entropy Loss of the active branch for x_t . The second part is a regularization term which is used to constrain the update of the parameters of the trunk for the learned concepts. λ is a hyperparameter used to adjust the importance of the new concept compared with the learned concepts, N_g represents the number of nodes from the root node to the growing point on the trunk and $\odot$ represents the Hadamard product. $\theta_t^{(j_0,k)}$ is a vector that contains the connection weights of the hidden layer in the k th node on the trunk. Once concept drift is detected, record $\theta_t^{(j_0,k)}$ as $\theta^{*(j_0,k)}$, $\forall k = 1, \dots, N_g$. $F^{(j_0,k)}$ denotes the Fisher information of each element in $\theta^{*(j_0,k)}$.

Fisher information is employed to measure the importance of the parameters of the trunk to the concept it has learned. How to compute $F^{(j_0,k)}$? Generally, with a given parameter θ , the prediction for a sample x can be defined as $p_\theta(y|x)$. According to Pascanu et al. [45], the Empirical Fisher Information Matrix F for θ is defined as:

$$F = E_{(x,y) \sim D} \left[\left(\frac{\partial \log p_\theta(y|x)}{\partial \theta} \right) \left(\frac{\partial \log p_\theta(y|x)}{\partial \theta} \right)^T \right] \quad (4)$$

where D means instance domain. Since $F \in \mathbb{R}^{d_\theta \times d_\theta}$ and d_θ denotes the number of parameters, and F is usually with millions of elements for neural networks, it is practically infeasible to store F . To handle this problem, Kirkpatrick et al. [44] assumed these parameters to be independent of each other and only used the diagonal elements of F to approximate F , which equals the square of the first derivative of the loss function to θ . The diagonal elements of F can be reshaped as a vector.

$F^{(j_0,k)} (\forall k = 1, \dots, N_{j_0})$ is initialized as a zero vector, every time concept drift is detected, $F^{(j_0,k)} (\forall k = 1, \dots, N_g)$ is updated as the historical average of the square of the gradients of $\theta^{(j_0,k)}$:

$$F^{(j_0,k)} = \frac{Q^{(k)}}{T^{(k)}}, \forall k = 1, \dots, N_g \quad (5)$$

where $Q^{(k)}$ represents the cumulative value of the gradient square of $\theta^{(j_0,k)}$, and $T^{(k)}$ represents the times of $\theta^{(j_0,k)}$ has been trained. When the trunk grows a new node, the initial values of the new node's $Q^{(k)}$ and $T^{(k)}$ are set as 0. When the backpropagation flow of x_t passes through $\theta^{(j_0,k)}$, $T^{(k)} \leftarrow T^{(k)} + 1, \forall k = 1, \dots, N_g$, $Q^{(k)}$ is updated to:

$$Q^{(k)} \leftarrow Q^{(k)} + \left(\frac{\partial L_t}{\partial \theta_t^{(j_0,k)}} \odot \frac{\partial L_t}{\partial \theta_t^{(j_0,k)}} \right)^T, \forall k = 1, \dots, N_g \quad (6)$$

According to the formula (3), the parameters with larger Fisher information will be subject to greater constraints when updated, to protect important parameters for the learned concepts and reduce the impact of learning a new concept on the classification performance for the learned concepts. On the other hand, the parameters with smaller Fisher information are less constrained when updated, allowing these parameters to adapt to the new concept quickly.

3.4.2. Concept drift detection

DDM [26] detects concept drifts based on the error rate on a window with variable length. To improve detection accuracy, in this paper, Cross Entropy Loss is used to substitute classification error (0 or 1), for the reason that classification loss contains more information about the difference between the true label and the predicted label. In addition, We have found in experiments that as the starting point of the window for DDM is fixed, the detection sensitivity will decrease as the window increases.

To improve sensitivity of detection, we utilize two sliding windows with different sizes to detect concept drifts. The lengths of these two windows are fixed, and the endpoint of each window corresponds to the latest sample. The larger window is used to calculate the overall loss, while the smaller window is used to calculate the current loss. If the current loss is significantly higher than the overall loss, it is considered that a concept drift has occurred. Specifically, $W_o^{(j)}$ is used to store the loss of j th branch for each sample in the larger window. The overall

average loss $p_o^{(j)}$ and the overall standard deviation $s_o^{(j)}$ are calculated from $W_o^{(j)}$, respectively. When $p_o^{(j)} + s_o^{(j)} < p_{min}^{(j)} + s_{min}^{(j)}$, $p_{min}^{(j)}$ and $s_{min}^{(j)}$ will record $p_o^{(j)}$ and $s_o^{(j)}$, respectively. $W_r^{(j)}$ is used to store the loss of j th branch for each sample in the smaller window. The current average loss $p_r^{(j)}$ and the current standard deviation $s_r^{(j)}$ are calculated from $W_r^{(j)}$, respectively.

When concept drift occurs, the current loss will rapidly increase, making it sensitive to concept drift. It should be noted that $p_{min}^{(j)}$, $s_{min}^{(j)}$, $p_r^{(j)}$ and $s_r^{(j)}$ are saved for each branch (including the trunk) separately during learning process.

- If $p_r^{(j^*)} + s_r^{(j^*)} < p_{min}^{(j^*)} + 2s_{min}^{(j^*)}$, no concept drift is detected. If $p_r^{(j^*)} + s_r^{(j^*)} \geq p_{min}^{(j^*)} + 2s_{min}^{(j^*)}$, warning stage starts. During the warning stage, the outputs of all branches (including the trunk) are calculated.
- Following learning process, if $p_r^{(j^*)} + s_r^{(j^*)} < p_{min}^{(j^*)} + 2s_{min}^{(j^*)}$ again, the warning stage will be eliminated, else if $p_{min}^{(j^*)} + 2s_{min}^{(j^*)} \leq p_r^{(j^*)} + s_r^{(j^*)} < p_{min}^{(j^*)} + 3s_{min}^{(j^*)}$, the warning state is maintained, and after a time if $p_r^{(j^*)} + s_r^{(j^*)} \geq p_{min}^{(j^*)} + 3s_{min}^{(j^*)}$, concept drift is detected, and the outputs of all branches (including the trunk) need to be used to determine whether a new concept or a recurring concept occurs.
- If the losses of all branches reach drift level: $p_r^{(j)} + s_r^{(j)} \geq p_{min}^{(j)} + 3s_{min}^{(j)}, \forall j = 1, \dots, H$, where H is the number of branches, it is considered that a new concept occur, and a new branch will be added to the trunk and set active. Otherwise, if $\exists j = 1, \dots, H, p_r^{(j)} + s_r^{(j)} < p_{min}^{(j)} + 3s_{min}^{(j)}$, it is considered that a recurring concept occurs, and the branch with the lowest $p_r^{(j)}$ is set active. Therefore, when encountering a recurring concept, ATNN employs the corresponding branch to learn the recurring concept and increase the depth of the branch, enabling ATNN to continuously improve its classification performance for the recurring concept.

3.4.3. Branch growing

ATNN determines when to grow a new branch on the trunk based on the result of concept drift detection. If a new concept is detected, a new branch containing only one node is added to the model and set active, and $\alpha^{(j^*,1)}$ is preset to 1. A key issue is where the new branch should be connected to the trunk, which determines how many parameters are shared between the new concept and the learned concepts. A suitable amount of shared parameters will make the model effectively utilize common knowledge between different concepts to help the model quickly learn the new concept and avoid negative transfer.

To learn the new concept as quickly as possible without affecting the classification performance for the learned concepts, Weighted Fisher regularization is used [44] as in the formula (3), and the gradient of the trunk's parameters G after concept drift is detected is used to measure the magnitude that the trunk's parameters need to be changed when a new concept is detected.

Assuming a concept drift warning occurs on the T_1 -th sample and then a new concept is detected on the T_2 -th sample, $[T_1, T_2]$ is concept drift warning stage and x_t ($T_1 \leq t \leq T_2$) is considered belonging to the new concept. $G^{(j_0,k)}$ ($\forall k = 1, \dots, N_{j_0}$) is calculated based on the loss of the trunk nodes $E_t^{(j_0,k)}$ during the concept drift warning stage:

$$G^{(j_0,k)} = \frac{1}{T_2 - T_1} \sum_{t=T_1}^{T_2} \left| \frac{\partial E_t^{(j_0,k)}}{\partial \theta^{(j_0,k)}} \right|^\top, \forall k = 1, \dots, N_{j_0} \quad (7)$$

Then, we calculate the cosine similarity $S_{cos}^{(j_0,k)}$ ($\forall k = 1, \dots, N_{j_0}$) between $F^{(j_0,k)}$ and $G^{(j_0,k)}$ and select the node with the lowest $S_{cos}^{(j_0,k)}$ as the location where the new branch should grow.

According to the formula (5) and (7), for the node with the lowest $S_{cos}^{(j_0,k)}$, the connection weights of its hidden layers with higher Fisher information correspond to relatively smaller change magnitude, indicating that they can be applied to new concept without requiring significant alteration. The connection weights of its hidden layer with smaller Fisher information correspond to a relatively larger change

magnitude, allowing for rapid learning of the new concept, updating these parameters without significantly affecting the classification performance for the learned concepts. Therefore, growing a new branch on the position found by the proposed method above can make the model quickly learn the new concept with minimal impact on the classification performance for the learned concepts.

Very interestingly, we have found two special situations in the experiment. If the correlation between the new concept and the corresponding concept which the trunk has learned is low, sharing nodes may lead to negative transfer [46]. In this situation, we found that $S_{cos}^{(j_0,k)}$ of all trunk's nodes are all at a high level. If the new concept and the corresponding concept which the trunk has learned are highly correlated, sharing more nodes can achieve good performance on the active branch. In this situation, we found that $S_{cos}^{(j_0,k)}$ of multiple trunk's nodes are at a low level. Thus, in this paper, two thresholds δ_1, δ_2 , ($\delta_1 > \delta_2$) are set to well select the growing point. If $S_{cos}^{(j_0,k)} > \delta_1, \forall k = 1, \dots, N_{j_0}$, the new branch will grow on the root node, which means that the new branch does not share nodes with the trunk. If $\exists k = 1, \dots, N_{j_0}, S_{cos}^{(j_0,k)} < \delta_2$, the deepest node among the nodes that meet this condition is selected as the position to grow a new branch.

ATNN adds a new branch or switches its active branch based on the result of concept drift detection. However, it should be noted that error of concept drift detection can lead to ATNN incorrectly addressing concept drift. If a concept drift occurs but is not detected, it can cause the knowledge of the active branch learned to be covered by another concept, leading to catastrophic forgetting. If a recurring concept is recognized as a new concept, it can cause ATNN to grow an unnecessary branch. In this paper, depending on the specifics of streaming data, the confidence level of concept drift detection and the window size can be adjusted to balance the risks of sensitivity and accuracy in concept drift detection.

In the formula (3), if the value of λ is preset too small, $\theta^{(j_0,k)}$ may deviate too much from $\theta^{*(j_0,k)}$, $\forall k = 1, \dots, N_g$, leading ATNN to misidentify a recurring concept as a new concept and employ a new branch to learn the recurring concept. To mitigate this problem, it is possible to choose a large value of λ to impose stricter constraints on updating the parameters of the trunk's nodes.

In addition, when new concepts or gradual drift occur continuously in the data stream, ATNN may experience excessive branch growth. ATNN introduces a strategy for pruning branches to avoid potential issues of excessive branch growth. When the number of branches exceeds the preset upper limit, the oldest branch, excluding the trunk, is removed from the model.

3.5. Parameter updating

The parameters involved in the active branch and trunk training include the output layer connection weights of the nodes on the active branch $\omega^{(j^*,k)}$, the hidden layer connection weights of the nodes on the active branch $\theta^{(j^*,k)}$, and the hidden layer connection weights of the nodes from the root to the growing point on the trunk $\theta^{(j_0,k)}$.

ATNN employs online gradient descent to update all connection weights. The updated rules are as follows:

$$\begin{aligned} \omega_{t+1}^{(j^*,k)} &\leftarrow \omega_t^{(j^*,k)} - \eta \frac{\partial L_t}{\partial \omega_t^{(j^*,k)}} \\ &= \omega_t^{(j^*,k)} - \eta \alpha_t^{(j^*,k)} \frac{\partial E_t^{(j^*,k)}}{\partial \omega_t^{(j^*,k)}}, \forall k = 1, \dots, N_{j^*} \end{aligned} \quad (8)$$

$$\begin{aligned} \theta_{t+1}^{(j^*,k)} &\leftarrow \theta_t^{(j^*,k)} - \eta \frac{\partial L_t}{\partial \theta_t^{(j^*,k)}} \\ &= \theta_t^{(j^*,k)} - \eta \sum_{p=k}^{N_{j^*}} \alpha_t^{(j^*,p)} \frac{\partial E_t^{(j^*,p)}}{\partial \theta_t^{(j^*,k)}}, \forall k = 1, \dots, N_{j^*} \end{aligned} \quad (9)$$

$$\begin{aligned}
\theta_{t+1}^{(j_0,k)} &\leftarrow \theta_t^{(j_0,k)} - \eta \frac{\partial L_t}{\partial \theta_t^{(j_0,k)}} \\
&= \theta_t^{(j_0,k)} - \eta \left[\sum_{p=k}^{N_{j^*}} \alpha_t^{(j^*,p)} \frac{\partial E_t^{(j^*,p)}}{\partial \theta_t^{(j_0,k)}} \right. \\
&\quad \left. + \lambda F^{(j_0,k)} \odot (\theta_t^{(j_0,k)} - \theta^{*(j_0,k)}) \right], \forall k = 1, \dots, N_g
\end{aligned} \quad (10)$$

3.6. Model complexity

An ATNN model should store the connection weights of all nodes for classification and additionally store $F^{(j_0,k)}$ and $\theta^{*(j_0,k)}$ for selecting a position on the trunk to grow a branch. Each node contains a hidden layer and an output layer, the hidden layer contains n^2 connection weights, and the output layer contains nc connection weights, where n is the width of hidden layer and c is the number of categories. For an ATNN model with H branches, the total number of nodes is $\sum_{j=1}^H N_j$, so the total number of connection weights is $\sum_{j=1}^H N_j(n^2 + nc)$. In a trunk node, the number of elements in $F^{(j_0,k)}$ and $\theta^{*(j_0,k)}$ is both n^2 . Therefore, the space complexity of an ATNN model is $O(2N_{j_0}n^2 + \sum_{j=1}^H N_j(n^2 + nc))$.

The time consumption of an ATNN model during training is mainly attributed to forward propagation, backpropagation, and updating connection weights. Each process needs to traverse N_g nodes on the trunk and all nodes on the active branch. For nodes on the trunk, only the hidden layers participate in computation, with a time consumption of $O(N_g n^2)$. For nodes on the active branch, both the hidden layers and the output layers participate in computation, with a time consumption of $O(N_{j^*}(n^2 + nc))$. Therefore, the time complexity of an ATNN model to learn a sample is $O(3N_g n^2 + 3N_{j^*}(n^2 + nc))$.

4. Experiments

We conducted five experiments to assess the effectiveness of the ATNN method across the following dimensions:

Classification performance comparison: We evaluated whether ATNN outperforms the existing state-of-the-art traditional methods and the DNN-based methods designed for data stream classification by examining 5 performance metrics: final cumulative accuracy, F1-score, Matthews correlation coefficient (MCC), final rank, and the number of parameters (NoP) on commonly used datasets. In addition, we evaluated whether ATNN can effectively overcome catastrophic forgetting by analyzing the changes of real-time accuracy when encountering recurring concepts.

Effectiveness of depth adaption: We compare the final cumulative accuracy (%) of the HBP models with different preset depths and the ATNN model to determine whether ATNN can find the optimal depth.

Effectiveness of adaptive position selection for branch growing: We try to fix branches at different positions on the trunk of an ATNN model which contains only one trunk to find the true optimal position for branch growing. We want to know whether the true optimal position found is round at the position selected by ATNN. Another question we want to know is whether growing branch at any position has a less significant impact on classification performance of the trunk.

Effectiveness of concept drift detection method: We replaced the concept drift detection module in ATNN with several well-known and widely used concept drift detection methods and compared the final cumulative accuracy on different datasets to analyze the effectiveness of ATNN's concept drift detection method.

Sensitivity analysis of learning rate: We evaluated the cumulative accuracy of the ATNN model employing different learning rates on different datasets and analyzed the sensitivity of ATNN's classification performance to learning rates.

Algorithm 1: ATNN

```

1 Input:  $\{(x_t, y_t), t = 1, \dots, T\}$ : Streaming data;  $\eta$ : Learning rate;  $\beta$ :
   Weight discounting parameter;  $s$ : Smoothing parameter;  $\lambda$ :
   Regularization hyperparameter;
2 Output: Prediction labels  $\{\hat{y}_t, t = 1, \dots, T\}$ ;
3 Initialize:  $\alpha^{(j^*,k)} = \frac{1}{N_{j^*}}$ ,  $Q^{(k)} = 0$ , where  $k = 1$ ;  $j_0 = j^* = 1$ ,  $N_{j^*} = 1$ ,
    $N_g = N_{j_0}$ ;
4 foreach  $t = 1, \dots, T$  do
5   Receive instance  $x_t$ ;
6   Output  $\hat{y}_t = F(x_t)$  by the formula (1);
7   Receive the true label  $y_t$ ;
8   Detect concept drift;
9   if no concept drift then
10    Update connection weights using online gradient descent;
11    Update  $\alpha^{(j^*,k)}, \forall k = 1, \dots, N_{j^*}$  by the formula (2);
12    if  $\alpha^{(j^*,N_{j^*})} = \max(\alpha^{(j^*,k)}, \forall k = 1, \dots, N_{j^*})$  then
13      Add a new node behind the deepest node,  $N_{j^*} \leftarrow N_{j^*} + 1$ ;
14    end
15    Update  $Q^{(k)}, k = 1, \dots, N_g$  by the formula (6);
16  end
17  if during concept drift warning stage then
18    Calculate the output of all branches;
19    Update  $G^{(j_0,k)}, \forall k = 1, \dots, N_{j_0}$  by the formula (7);
20  end
21  if new concept then
22    Update  $F^{(j_0,k)}, \forall k = 1, \dots, N_g$  by the formula (5);
23    Calculate  $S_{cos}^{(j_0,k)}, \forall k = 1, \dots, N_{j_0}$ ;
24    Add a new branch to a node on the trunk with the lowest
        $S_{cos}^{(j_0,k)}$ ;
25    Set the new branch as the active branch;
26    Update  $N_g$ ;
27  end
28  if recurring concept then
29    Update  $F^{(j_0,k)}, \forall k = 1, \dots, N_g$  by the formula (5);
30    Activate the branch corresponding to the recurring concept;
31  end
32 end

```

4.1. Datasets

12 widely used datasets for data stream classification are employed in this paper. In the real datasets of Covtype, Weather, and Electricity, the types of concept drift are not known, while the datasets of MNIST and FMNIST and the artificial datasets of SCD, Tree, RBF, LED, MIXED, Agrawal, and Sine contain sudden drifts recurring concepts. The method of generating artificial datasets is to randomly initialize sample features, and then input the samples into predefined probability distribution functions or randomly initialized models to obtain sample labels. Concept drift is simulated by selecting different predefined probability distribution functions or using different random seeds to initialize models. The concepts change once every 5000 samples and change 3 times periodically, such as C1-C2-C1-C2-C1-C2. The details of the datasets are shown in Table 1.

- Covtype [47], a real dataset from UCI repository. Classification of pixels into 7 forest cover types is based on the attributes such as elevation, aspect, slope, hillshade, soil type, and more.
- Weather [48], a real dataset that records daily meteorological data, including temperature, pressure, wind speed, etc., from 1949 to 1999 at the Offutt Air Force Base in Bellevue, Nebraska. Class labels are based on the binary indicator(s) of whether the day is raining.
- Electricity [49], a real dataset that contains electricity prices in New South Wales, Australia, affected by weather, user demand, supply conditions, seasons, and other factors.

- MNIST [50], a well-known dataset of handwritten digits. We manually added concept drifts to this dataset. It contains two concepts, the concept 1 is the original data, while in the concept 2, the labels of 0, 2, and 4 are exchanged with the labels of 1, 3, and 5, respectively.
- FMNIST, it contains four concepts, the concept 1 is the original data of the well-known FashionMNIST [51], the other three concepts are generated by rotating each of the original data 90 degrees, 180 degrees, and 270 degrees, respectively.
- SCD, it is an artificial dataset used by Sahoo et al. in the study of HBP [12]. After inputting samples into a randomly initialized multi-layer perceptron (MLP) model whose depth is 8, and the width of the hidden layer is 100, the labels predicted by the model are used as the true labels for the samples. Two concepts are obtained by using two MLP models with different randomly initialized connection weights to generate labels.
- Tree [52], the label of each sample in this dataset is generated by inputting the sample into a decision tree model which is constructed by choosing attributes at random to split and assigning a random class label to each leaf, the first and second layers of the decision tree have no leaves, 50% of the nodes in the third and fourth layers are leaves, and the fifth layer is all leaves. Two concepts are obtained by using two decision tree models with different random split attributes to generate labels.
- RBF [52], 100 random centroids are generated through random radial basis functions, with each centroid randomly assigned to one of 10 categories. A new sample is created by choosing one of the centroids at random and offsetting the attributes at a random direction from the centroid's center. Three concepts are generated by initializing 100 centroids using three different random seeds.
- LED [53], this dataset contains data used to predict the numbers on the seven-segment LED display. It consists of 10 original samples with different categories, each with 7 basic attributes. A new sample of the concept 1 is generated by randomly selecting an original sample and adding 5% attribute noise and 17 random irrelevant features to it. Based on the concept 1, the concept 2 is obtained by randomly selecting a base attribute and exchanging positions with an irrelevant attribute, while the concept 3 is obtained by randomly selecting 4 base attributes and exchanging positions with 4 irrelevant attributes.
- MIXED [26], this dataset contains two boolean attributes v, w , and two numeric attributes x, y uniformly distributed in $[0,1]$. In concept 1, the examples are classified as positive if two of three conditions are satisfied: $v, w, y < 0.5 + 0.3\sin(3\pi x)$; otherwise, they are classified as negative. The concept 2 is obtained by reversing the class labels of the concept 1.
- Agrawal [54], this dataset contains six numeric features and three categorical features. There are ten predefined functions used to determine binary class labels of samples based on their features. The experiment uses three functions as three concepts.
- Sine [26], this dataset has two attributes. Each attribute has values uniformly distributed in $[0, 1]$. Three concepts are generated in this dataset, with the discriminants for the positive label satisfying $y < 0.5 + 0.3\sin(3\pi x)$, $y < \sin(x)$, or $y > \sin(x)$, respectively.

We repeat each experiment 10 times and calculate the average evaluation metrics to ensure the reliability of the results. In all the experiments, the network parameters are initialized randomly, the artificial datasets are regenerated using different random seeds, while for the datasets of Coverttype, Weather, and Electricity, 90% instances of each dataset are randomly sampled without changing their order. For the datasets of MNIST and FMNIST, after manually adding concept drifts to the original dataset, 90% of the samples were randomly selected from the original datasets for each experiment.

4.2. Baselines

In this paper, the proposed ATNN method is compared with several state-of-the-art traditional methods: CAND [9], BELS [55], AiRStream [25], and several state-of-the-art DNN-based methods: SEOA [11], HBP [12], ADL [16], IADM [14] and OLedRVFL [15].

DNN-based methods require preset hyperparameters such as width, depth, and learning rate. ADL adaptively adjusts the width and depth of its model during learning process. The widths for HBP, SEOA, OLedRVFL, and IADM are set to 256, while the depths for them are set to 8, because they require overcomplete model depth to be set before training, which is sufficient for all datasets. We use the values of 0.1, 0.05, 0.02, 0.01, 0.005, and 0.001 to test the learning rate of each DNN-based algorithm and select the most optimal learning rate. The other hyperparameter settings for each algorithm are as follows, and these hyperparameters are consistent across all datasets.

- CAND [9]. It is an ensemble of MLP with different learning rates (0.5, 0.05, 0.005, or 0.0005) and widths (256, 512, or 1024), designed to tackle the challenge of optimizing hyperparameters for MLP in data stream scenarios. Due to the component classifier employed in CAND in our experiments is MLP with single hidden layer, CAND is considered as a traditional method. The size of the classifier pool is set to 30.
- BELS [55]. It is proposed as an extension of BLS [56], with a model structure consisting of a hidden layer connected to multiple output layers, addressing the challenges associated with concept drifts by utilizing an ensemble of multiple output layers. The width of the hidden layer is set to 256, and the size of the classifier pool is set to 30.
- AiRStream [25]. It employs Hoeffding Tree [57] as component classifiers and determines whether to build a new classifier or reuse an old Hoeffding Tree based on concept drift detection. It proposes a method of detecting and repairing concept drift adaptation errors to enhance the algorithm's robustness. The window size for concept drift detection is set to 1500. The number of inactive models tested at each drift repair step is set to 5. The repair sensitivity is 0.2 and the repair period is 2000.
- HBP [12]. HBP proposes a structure where each hidden layer is connected to an output layer and utilizes weighted ensemble of the output of each layer to classify. Learning rate $\eta = 0.01$, the discounting factor $\beta = 0.99$ and the smoothing factor $s = 0.2$.
- SEOA [11]. SEOA improves the model's adaptability to concept drifts based on HBP. The improvements encompass replacing hidden layers with adaptive depth units and employing selective ensemble classification. Learning rate $\eta = 0.001$, discounting factor $\beta = 0.99$, the model fluctuation determination thresholds $\theta_1 = 0.2$ and $\theta_2 = 0.05$.
- ADL [16]. ADL employs a structure similar to HBP. It dynamically increases the depth of the model based on concept drift detection and reduces or increases the width of the hidden layer based on whether the model is overfitting or underfitting. Learning rate $\eta = 0.02$, the threshold $\delta = 0.05$, and the step size $\zeta = 0.001$.
- IADM [14]. IADM uses a structure similar to HBP. It employs weighted Fisher regularization to improve the stability of hidden neurons and utilizes attention networks to adjust the weights of classifiers. Learning rate $\eta = 0.01$, regularization parameter $\lambda = 1800$.
- OLedRVFL [15]. It employs a multi-layer Random Vector Functional Link network (RVFL [58]), randomly initializes hidden layer parameters, and updates output layer parameters using recursive regularized least squares.
- ATNN. The proposed method. Learning rate $\eta = 0.02$, weight discounting parameter $\beta = 0.99$, smoothing factor $s = 0.2$, regularization parameter $\lambda = 5000$, threshold $\delta_1 = 0.85$, $\delta_2 = 0.7$, and window size $W_o = 500$, $W_r = 50$.

Table 1
Dataset.

Datasets	Instances	Features	Class	Concept drifts	Types
Coverttype	581 K	54	7	–	Not known
Weather	18 K	8	2	–	Not known
Electricity	45 K	14	2	–	Not known
MNIST	30 K	784	10	1-2-1-2-1-2	Sudden, recurring
FMNIST	60 K	784	10	1-2-3-4-1-2-3-4-1-2-3-4	Sudden, recurring
SCD	30 K	25	10	1-2-1-2-1-2	Sudden, recurring
Tree	30 K	15	4	1-2-1-2-1-2	Sudden, recurring
RBF	45 K	10	10	1-2-3-1-2-3-1-2-3	Sudden, recurring
LED	45 K	24	10	1-2-3-1-2-3-1-2-3	Sudden, recurring
MIXED	30 K	4	2	1-2-1-2-1-2	Sudden, recurring
Agrawal	45 K	9	2	1-2-3-1-2-3-1-2-3	Sudden, recurring
Sine	45 K	4	2	1-2-3-1-2-3-1-2-3	Sudden, recurring

Table 2

Final cumulative accuracy (%) of different methods.

Datasets	Traditional methods			DNN-based methods					
	BELS	AirStream	CAND	ADL	OLedRVFL	HBP	SEOA	IADM	ATNN
Coverttype	85.7•	87.8•	92.0	73.0•	77.1•	91.3•	89.0•	90.1•	92.7
Weather	78.5	75.0•	77.0•	73.4•	78.3	74.8•	75.1•	72.5•	78.1
Electricity	81.7•	85.4	84.6	66.0•	69.4•	78.5•	73.7•	74.2•	83.6
MNIST	78.2•	60.5•	88.5	82.9•	65.2•	85.4•	87.2•	81.0•	89.1
FMNIST	71.0•	61.3•	75.4•	67.0•	62.9•	73.6•	73.2•	71.1•	76.6
SCD	53.8•	58.8•	67.0•	63.4•	50.5•	61.0•	65.6•	57.3•	73.6
Tree	82.3•	82.7•	84.8•	76.2•	67.0•	78.5•	84.4•	72.6•	88.0
RBF	66.7•	50.0•	69.5•	35.7•	44.3•	50.2•	71.8•	42.9•	82.6
LED	77.9•	88.9•	91.2•	86.4•	72.0•	83.4•	88.1•	82.2•	93.7
MIXED	87.9•	88.6•	90.0•	85.2•	51.0•	86.5•	90.4•	84.7•	92.8
Agrawal	79.6•	85.8	85.0	82.7•	76.0•	82.8•	84.5•	80.5•	85.9
Sine	76.4•	88.5•	91.1	90.3•	65.1•	90.4•	91.3	90.7•	91.4
Final rank	5.9	5.3	2.5	6.8	7.9	5.0	3.7	6.6	1.3
NoP	76.3 K	–	280.1 K	32.5 K	407.5 K	465.7 K	930.3 K	1385.5 K	1596.4 K

4.3. Results and discussion

4.3.1. Classification performance

Table 2 shows the final cumulative accuracies of ATNN and the other methods. The paired t-test at a 95% confidence level has been conducted to detect whether the differences between ATNN and each baseline method are statistically significant, and • indicates ATNN is significantly better than the corresponding baseline method. It can be observed from Table 2 that ATNN achieves the optimal cumulative classification accuracy and has significant advantages over the other methods except on the datasets of Weather and Electricity.

We use the Friedman test to conduct statistical tests on the performance analysis among the comparison methods. According to our experimental results, the calculated $F_{(8,88)} = 58.8$, the critical value of $F_{(8,88)}$ at significance level $\alpha = 0.05$ is 2.045, so the null hypothesis that there is no significant difference in the average accuracy of all methods is rejected. The critical difference (CD) of all methods is calculated by the Bonferroni–Dunn test to show the relation of the final cumulative accuracy between all methods. The performances of the two methods are significantly different if the corresponding average rank differs by at least the critical difference. At the 0.05 significance level, $CD = 3.04$. These results of the statistical analysis are shown in Fig. 3. The results show that the final cumulative accuracy of ATNN is significantly better than that of ADL, OLedRVFL, IADM, HBP, BELS, and AirStream.

Due to ATNN's multi-branch structure, its NoP is more than that of all baselines. However, most of the time, only the trunk and the active branch of ATNN are involved in calculation, and the classification results of the other branches are only needed during the warning stage. Therefore, the other branches can be stored in external memory and read in when needed, which can greatly improve the spatial efficiency of ATNN.

In addition, we compared the F1-score and MCC of ATNN and the other methods. Tables 3 and 4 respectively show the experimental results. Like in Table 2, the paired t-test at a 95% confidence level has

been conducted to detect whether the differences between ATNN and each baseline method are statistically significant, and • indicates ATNN is significantly better than the corresponding baseline method. From Tables 3 and 4, it can be observed that ATNN achieves the optimal F1-score and MCC, and has significant advantages over the other methods except on the datasets of Weather and Electricity.

In Fig. 4, the real-time accuracy is used to track the classification process of ATNN and the other methods. On the real datasets of Coverttype, Electricity, and Weather, the real-time accuracies of ATNN are always higher and more stable than most baselines. On the artificial datasets and the datasets of MNIST and FMNIST, once encountering concept drifts, the real-time accuracy of all methods shows a significant decrease, however, ATNN can quickly recover to its original level of classification accuracy, while the real-time accuracies of the other baseline methods improve slowly. This is because when a new concept occurs, a new branch for the new concept grows on the trunk, and sharing nodes can make the new branch fast converge.

In Fig. 4, once recurring concept occurs, the real-time accuracy of ATNN first significantly decreases and then rapidly increases to the original level for the recurring concept and continues to improve. This is because when encountering recurring concept, the real-time accuracy of ATNN decreases due to a delay in concept drift detection. After a recurring concept is detected, ATNN switches to the branch which corresponds to the recurring concept detected and activates and trains it. Thus, the branch can continue to learn the recurring concept and then continuously improve its classification accuracy, while the baseline algorithms need to relearn the recurring concept. Thus, ATNN can effectively overcome catastrophic forgetting.

4.3.2. Effectiveness of depth adaption

During learning process, each branch of an ATNN model only learns one concept and adaptively increases its depth. Therefore, for simplicity, only the samples of one concept in each dataset are selected in the experiment. To find the optimal depth for the HBP model on each

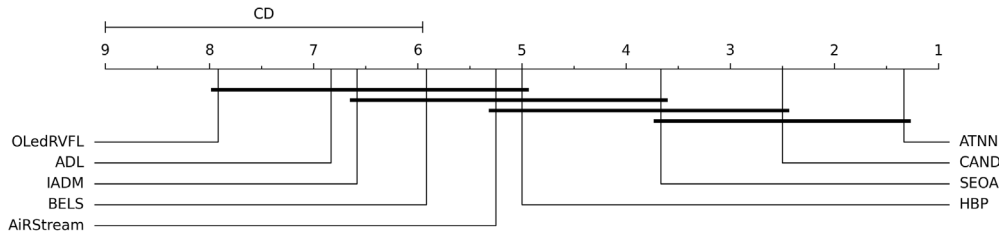


Fig. 3. Comparison of final cumulative accuracy with the Bonferroni-Dunn test.

Table 3

F1-score of different methods.

Datasets	Traditional methods			DNN-based methods					
	BELS	AiRStream	CAND	ADL	OledRVFL	HBP	SEOA	IADM	ATNN
Coverttype	0.70	0.77	0.87	0.58	0.62	0.83	0.80	0.82	0.87
Weather	0.74	0.70	0.72	0.65	0.73	0.66	0.67	0.64	0.73
Electricity	0.79	0.85	0.84	0.65	0.68	0.77	0.72	0.73	0.83
MNIST	0.78	0.56	0.88	0.83	0.64	0.85	0.87	0.81	0.89
FMNIST	0.70	0.60	0.75	0.67	0.62	0.73	0.73	0.71	0.76
SCD	0.49	0.57	0.65	0.62	0.47	0.60	0.64	0.55	0.72
Tree	0.81	0.82	0.84	0.75	0.65	0.78	0.84	0.72	0.87
RBF	0.66	0.48	0.69	0.33	0.43	0.50	0.71	0.42	0.82
LED	0.78	0.89	0.91	0.86	0.72	0.83	0.88	0.82	0.93
MIXED	0.88	0.89	0.90	0.85	0.51	0.87	0.90	0.85	0.92
Agrawal	0.76	0.84	0.84	0.81	0.72	0.81	0.83	0.79	0.84
Sine	0.76	0.89	0.91	0.90	0.65	0.90	0.91	0.91	0.91

Table 4

MCC of different methods.

Datasets	Traditional methods			DNN-based methods					
	BELS	AiRStream	CAND	ADL	OledRVFL	HBP	SEOA	IADM	ATNN
Coverttype	0.65	0.78	0.87	0.56	0.60	0.85	0.82	0.85	0.89
Weather	0.47	0.40	0.45	0.32	0.47	0.34	0.37	0.33	0.46
Electricity	0.62	0.70	0.69	0.31	0.42	0.54	0.45	0.47	0.66
MNIST	0.77	0.59	0.87	0.81	0.61	0.83	0.86	0.80	0.88
FMNIST	0.68	0.56	0.72	0.62	0.59	0.71	0.70	0.68	0.73
SCD	0.47	0.53	0.63	0.59	0.43	0.56	0.61	0.52	0.70
Tree	0.75	0.77	0.80	0.64	0.54	0.71	0.79	0.64	0.83
RBF	0.63	0.42	0.66	0.26	0.40	0.45	0.67	0.36	0.80
LED	0.76	0.88	0.90	0.85	0.70	0.82	0.87	0.81	0.93
MIXED	0.74	0.75	0.79	0.71	0.09	0.73	0.80	0.68	0.85
Agrawal	0.54	0.69	0.69	0.61	0.43	0.63	0.66	0.58	0.69
Sine	0.53	0.76	0.82	0.81	0.38	0.81	0.83	0.82	0.83

dataset, the final cumulative accuracies (%) of the HBP models with different preset depths are computed. Then, the optimal depth found on each dataset is compared with the final depth of the ATNN model trained on the same dataset. Thus, we can determine whether ATNN can find the optimal depth. In this experiment, the HBP models with different depths take the depths from 2 to 10, respectively.

For the artificial datasets and the datasets of MNIST and FMNIST, 30000 samples from their first concept are sampled for the experiment. Due to concept drift unknown in the datasets of Coverttype, Weather, and Electricity, their complete datasets are used. The experiment on each dataset is repeated 10 times and the average final cumulative accuracies are presented in Table 5. Bold represents the highest accuracy, and the number to the right of the last column is the final depth of the trained ATNN model.

According to Table 5, the optimal depth for an HBP model varies from different datasets. For an HBP model, when the preset depth exceeds the corresponding optimal depth, the classification accuracy decreases as its preset depth increases. These results show in fact that it is difficult to preset a suitable depth for an HBP model. Here, it should be noted that this interesting observation for HBP is not mentioned in the paper [12]. The reason behind this observation is that when using gradient descent to update the HBP model, the gradients of shallow nodes are calculated by backpropagation of multiple deep

classifiers. According to Ashfahani et al. [16], excessive depth will lead to overfitting and reduce generalization performance. Thus, if the depth of an HBP model is preset over large, it may include multiple deep classifiers with poor classification performance, and the backpropagation of gradient will interfere with the convergence of shallow networks in an HBP model.

The strategy of adaptively increasing the branch's depth of an ATNN model can alleviate this problem. From Table 5, it can be observed that the final depth of the ATNN model is approximately equivalent to the optimal depth found by comparing the accuracies of the HBP model with different preset depths across all datasets. The Pearson correlation coefficient is employed to further analyze the relationship between them on each dataset. The Pearson correlation coefficient is 0.80, and $p = 1.8 \times 10^{-3} < 0.05$, these results illustrate that the final depth of the ATNN model and the optimal depth found by the HBP model are strongly correlated, indicating that the mechanism of depth adaption of ATNN is effective.

4.3.3. Effectiveness of adaptive position selection for branch growing

In this experiment, comparison models are obtained by fixing the position to grow a branch on the trunk of an ATNN model from the 0-th to the 6-th nodes. The trunk of an ATNN model is fixed to 8 layers. It should be noted that if a branch grows on the 0-th node on the

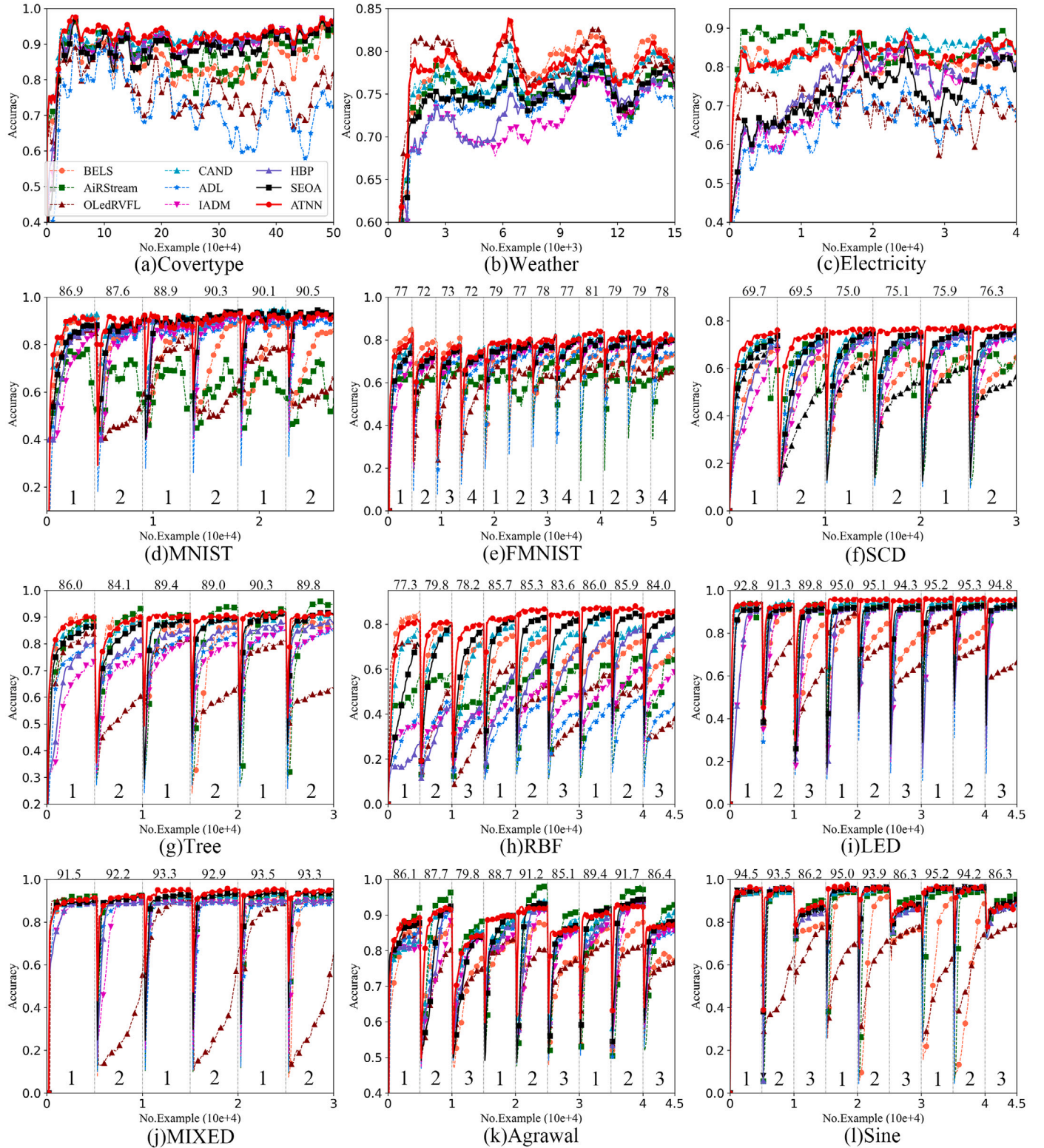


Fig. 4. Real-time accuracies of different methods. The numbers above the x-axis represent the concepts that the data belong to. The numbers on the top of each subfigure display the average real-time classification accuracy of the ATNN model during the corresponding segment of learning.

trunk, the new branch and the trunk do not share any nodes. In this experiment, the first two concepts of each artificial dataset are used. The samples of the first concept are used to train the trunk of an ATNN model, the first 50 samples of the second concept are used to calculate S_{cos} , and the remaining samples are used to train the branch on the trunk. Two corresponding separate test datasets are used to evaluate

the trunk and the branch, respectively. The size of each test dataset is set to 5000.

The experimental results are presented in Fig. 5. The first row presents the value of $S_{cos}^{(j_0,k)}$ of the k th layer of the trunk for different datasets. According to ATNN, the layer with the lowest value of $S_{cos}^{(j_0,k)}$ means the optimal position on which a branch should grow. The second

Table 5

Final cumulative accuracy (%) of the HBP model with different preset depths and the ATNN model.

Datasets	HBP model with different preset depths									ATNN model
	2	3	4	5	6	7	8	9	10	
Covertypes	89.0	90.8	91.1	91.7	92.0	91.8	91.6	91.3	91.0	92.3/6.8
Weather	74.5	76.0	77.2	76.9	76.3	76.1	75.9	75.6	75.5	78.1/4.5
Electricity	80.6	81.2	82.0	81.3	80.4	80.3	79.7	79.0	78.6	81.5/4.2
MNIST	90.3	90.8	90.8	90.4	90.1	90.0	89.5	89.4	89.0	91.1/3.7
FMNIST	79.3	80.2	80.9	81.3	80.8	80.8	80.2	79.5	79.3	81.0/4.9
SCD	74.1	73.4	72.2	72.0	71.4	71.2	71.1	70.6	70.0	74.3/3.0
Tree	90.7	91.3	91.3	91.1	91.1	91.0	90.8	90.7	90.6	91.4/3.8
RBF	83.4	84.1	85.3	85.2	84.0	83.6	83.0	82.7	82.5	85.1/4.3
LED	91.4	91.2	91.1	91.0	90.8	90.8	90.7	90.6	90.5	91.3/3.2
MIXED	94.1	94.7	95.0	94.5	94.4	94.1	93.9	93.8	93.6	94.6/4.2
Agrawal	83.4	84.2	84.3	84.1	83.5	83.1	82.8	82.5	82.4	84.3/3.7
Sine	95.6	95.9	96.0	96.1	96.1	95.9	95.7	95.7	95.5	95.9/4.2
Mean	85.5	86.1	86.4	86.2	85.9	85.7	85.4	85.1	84.9	86.7/4.2

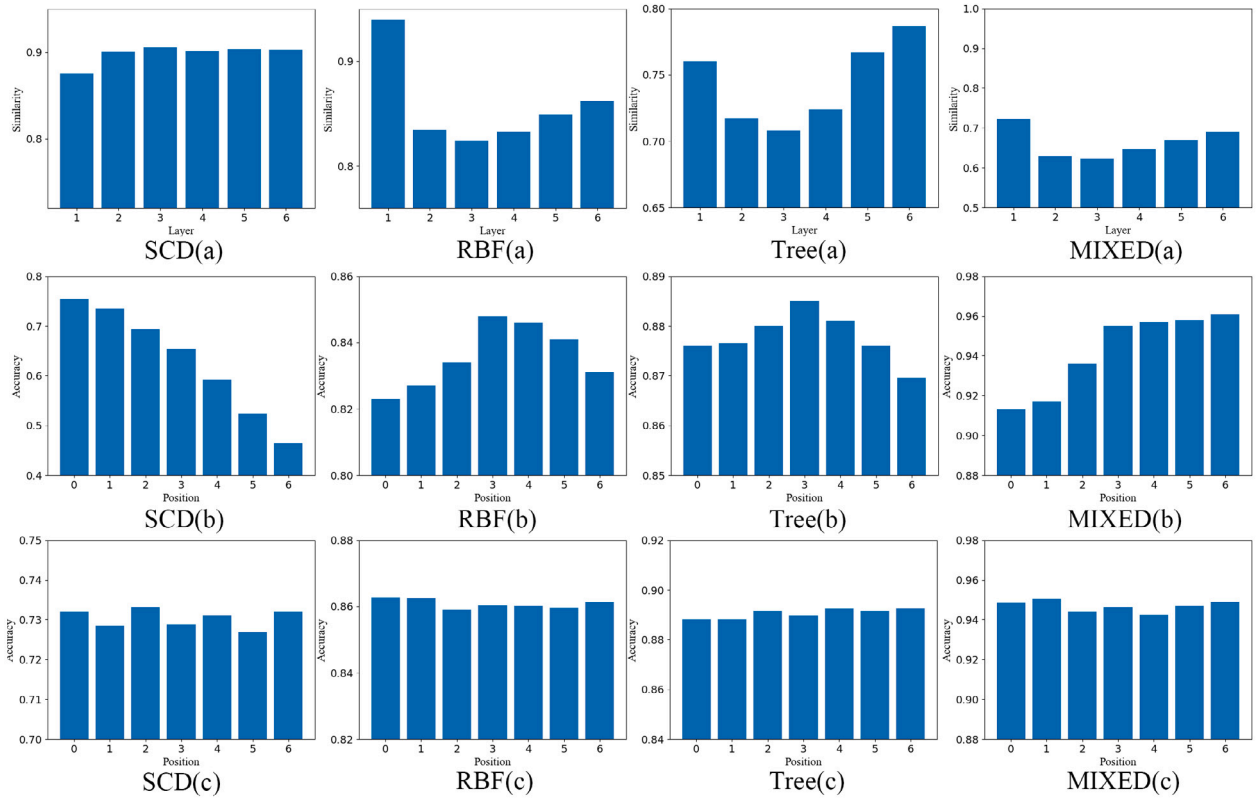


Fig. 5. First row: $S_{cos}^{(j_0,k)}$ of the k th layer of the trunk. Second row: the classification accuracies of the branches growing on different positions on the trunk. Third row: the classification accuracies of the trunk after branches are added at different positions.

row presents the classification accuracies of the branches growing on different positions on the trunk for different datasets. It can be observed from Fig. 5 that the branch which grows on the optimal position can get the best accuracy. The third row presents the classification accuracies of the trunk after a branch is added at different positions for different datasets. These results show that the classification accuracy of the trunk is less influenced by branch growing. This is due to regularization constraints on updating the connection weights of the hidden layer of the trunk, thus growing branches at any position has a less significant impact on the classification performance of the trunk.

To be more specific, the SCD dataset is generated by a randomly initialized MLP-8 model, different concepts use different random seeds, and the correlation between different concepts is low. Thus, in the experiment on SCD, sharing nodes will result in negative transfer, the optimal position to grow a branch is at 0. For the dataset of RBF and Tree, different concepts are generated based on certain rules, so there are some general shallow features between different concepts and

growing a new branch in the middle position on the trunk is better. For the dataset of MIXED, different concepts only involve label exchange. In this case, theoretically, adjusting the parameters of the output layer should be sufficient to adapt to concept drift. Experimental results indicate that $S_{cos}^{(j_0,k)}$ on multiple nodes are at a low level and more nodes need to be shared to achieve optimal classification accuracy.

To clarify the problem more clearly, Table 6 shows the classification accuracies of the branches growing at different positions on the trunk and the branch of a trained ATNN model on nine datasets. The number to the right of the last column presents the positions found by ATNN, on which a branch should grow. It can be observed from Table 6 that the growing position found by ATNN is round at the growing position on which the growing branch gets the best accuracy. Pearson correlation coefficient is used to analyze their relationship, and it is found that the Pearson correlation coefficient is 0.94, and $p = 1.3 \times 10^{-4} < 0.05$, which indicates a strong correlation between the growing position found by

Table 6

Classification accuracies (%) of the branches growing at different positions on the trunk and the branch of a trained ATNN model.

Datasets	Branch growing on different positions							ATNN
	0	1	2	3	4	5	6	
MNIST	89.1	89.3	89.4	89.5	89.4	89.1	88.4	89.4/2.8
FMNIST	63.4	68.7	71.3	71.0	70.2	67.9	67.4	71.2/2.5
SCD	75.4	73.4	69.3	65.3	59.2	52.3	46.4	75.4/0
Tree	87.6	87.7	88.0	88.5	88.1	87.6	87.0	88.3/2.5
RBF	82.3	82.7	83.4	84.8	84.6	84.1	83.1	84.4/2.8
LED	95.4	95.7	96.3	96.4	96.5	96.2	95.4	96.3/3.4
MIXED	91.3	91.7	93.6	95.5	95.7	95.8	96.1	95.9/5.7
Agrawal	91.1	91.8	90.9	90.0	89.1	87.4	85.1	91.0/2.0
Sine	95.1	95.4	95.9	95.9	96.0	95.9	95.9	95.8/2.9
Mean	85.3	86.3	86.5	86.3	85.4	84.0	82.7	87.3

Table 7

Final cumulative accuracy (%) of the ATNN models deploying different concept drift detection methods.

Datasets	ATNN employing different drift detection methods					ATNN
	AOIL	DDM	EDDM	HDDM	RDDM	
Coverttype	92.6	92.0	92.1	91.7	91.9	92.7
Weather	76.2	78.1	73.7	73.6	78.0	78.1
Electricity	83.5	84.1	83.4	83.1	83.6	83.6
MNIST	89.0	85.6	86.4	88.3	89.1	89.1
FMNIST	76.3	71.3	69.1	73.0	74.7	76.6
SCD	73.1	70.0	65.7	67.9	62.5	73.6
Tree	87.3	84.8	82.9	85.7	88.1	88.0
RBF	82.3	78.7	74.4	78.9	78.1	82.6
LED	93.4	92.1	91.2	92.5	93.7	93.7
MIXED	92.1	92.4	89.9	92.9	92.1	92.8
Agrawal	86.3	83.4	79.8	84.5	82.5	85.9
Sine	91.1	90.9	89.5	91.4	91.3	91.4
Mean	85.2	83.6	81.5	83.6	83.8	85.6

ATNN and the growing position on which the growing branch get the best classification accuracy.

4.3.4. Effectiveness of atnn's concept drift detection method

In this experiment, we replaced the concept drift detection module in ATNN with several well-known and widely used concept drift detection methods: DDM [26], EDDM [29], HDDM [30], RDDM [32], and AOIL [33]. We then compared the final cumulative accuracies of the ATNN models with different methods to detect concept drifts on different datasets to analyze the effectiveness of ATNN's concept drift detection method.

Table 7 shows the final cumulative accuracies of the ATNN model with different concept drift detection methods. It can be observed that the ATNN model employing our concept drift detection method can achieve the best final cumulative accuracy, EDDM detected too many false positives for concept drift during stationary periods, resulting in the ATNN model containing too many branches and poor classification performance. ATNN using DDM, HDDM, and RDDM to detect concept drift show similar classification accuracy. Similar to ATNN's detection method, AOIL is a loss-based detection method, with the difference being that AOIL detects concept drift based on changes in classification loss within a single window. AOIL exhibits classification accuracy similar to ATNN on multiple datasets.

4.3.5. Sensitivity analysis of learning rate

In this experiment, we evaluated the final cumulative accuracy of ATNN using different learning rates on different datasets, with learning different rates of 0.1, 0.05, 0.02, 0.01, 0.005, 0.002, and 0.001, respectively. Table 8 shows the experimental results. It can be observed that when the learning rate is set too high, such as 0.1 or 0.05, the model may struggle to converge and exhibit very poor classification performance on all datasets. If the learning rate is set too low, the model will adapt to concept drift slowly, which also leads to poor classification accuracy. In most cases, setting the learning rate to 0.02 is appropriate.

Table 8

Final cumulative accuracy (%) of the ATNN models using different learning rates.

Datasets	ATNN using different learning rates						
	0.1	0.05	0.02	0.01	0.005	0.002	0.001
Coverttype	90.5	91.3	92.7	93.1	92.7	92.5	92.2
Weather	73.1	73.1	78.1	77.8	77.6	77.3	77.2
Electricity	73.1	73.3	83.6	83.5	83.5	83.0	83.1
MNIST	53.6	70.9	89.1	88.3	87.5	86.2	85.9
FMNIST	69.8	73.2	76.6	76.6	76.2	74.4	74.4
SCD	49.2	66.1	73.6	72.8	70.3	67.5	65.3
Tree	70.1	83.1	88.0	86.7	85.8	84.4	84.2
RBF	53.2	72.9	82.6	80.7	77.5	72.0	70.2
LED	77.1	82.6	93.7	92.7	92.0	90.4	90.2
MIXED	79.3	86.7	92.8	91.4	90.3	90.5	89.4
Agrawal	73.7	77.6	85.9	83.1	82.5	81.2	81.2
Sine	79.9	86.1	91.4	91.1	90.8	90.6	90.6
Mean	70.2	78.1	85.6	84.8	83.9	82.5	82.0

5. Conclusion

In this paper, we find two key problems in DNN-based concept drift adaptation methods that have not been addressed. One is that it is difficult to set an appropriate depth for a DNN-based model in advance for streaming data classification. The second is the catastrophic forgetting problem caused by concept drifts making the DNN-based model learn recurring concepts slowly. The proposed adaptive tree-like neural network (ATNN) can effectively handle these two problems.

ATNN adaptively increases the depth of its branch when the weight of the deepest node in the branch becomes the largest, and once a new concept is detected, it chooses a suitable position on its trunk to grow a branch for a new concept according to the relation between Fisher information and gradient of parameters and activate the added branch. The rationality of ATNN to do these operations is validated in experiments. Adaptively increasing the depth of branches makes ATNN continuously learn recurring concepts while adaptively choosing a position on the trunk to grow a branch makes ATNN to fast learn new concepts. Experiments also illustrate that ATNN can more quickly adapt to concept drifts, and continuously improve classification performance for recurring concepts.

Future work includes how to improve ATNN to handle gradual drifts and incremental drifts, how to more accurately detect concept drifts using the multi-branch nature of ATNN, and how to prune ATNN.

CRedit authorship contribution statement

YiMin Wen: Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Xiang Liu:** Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation. **Hang Yu:** Writing – review & editing, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (62366011, 62302287), the Key R&D Program of Guangxi, China under Grant (AB21220023), and Guangxi Key Laboratory of Image and Graphic Intelligent Processing, China (GIIP2306).

References

- [1] B. Jiao, Y. Guo, C. Yang, J. Pu, Z. Zheng, D. Gong, Incremental weighted ensemble for data streams with concept drift, *IEEE Trans. Artif. Intell.* 5 (1) (2024) 92–103.
- [2] S.C. Hoi, D. Sahoo, J. Lu, P. Zhao, Online learning: A comprehensive survey, *Neurocomputing* 459 (2021) 249–289.
- [3] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, *IEEE Trans. Knowl. Data Eng.* 31 (12) (2018) 2346–2363.
- [4] G. Hulten, L. Spencer, P. Domingos, Mining time-changing data streams, in: *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2001, pp. 97–106.
- [5] G. Liu, H.-r. Cheng, Z.-g. Qin, Q. Liu, C.-x. Liu, E-CVFD: An improving CVFD method for concept drift data stream, in: *Proceedings of the 2013 International Conference on Communications, Circuits and Systems*, 2013, pp. 315–318.
- [6] W. Cao, Z. Ming, Z. Xu, J. Zhang, Q. Wang, Online sequential extreme learning machine with dynamic forgetting factor, *IEEE Access* 7 (2019) 179746–179757.
- [7] S. Ramírez-Gallego, B. Krawczyk, S. García, M. Woźniak, J.M. Benítez, F. Herrera, Nearest neighbor classification for high-speed big data streams using spark, *IEEE Trans. Syst. Man Cybern. Syst.* 47 (10) (2017) 2727–2739.
- [8] B. Krawczyk, L.L. Minku, J. Gama, J. Stefanowski, M. Woźniak, Ensemble learning for data stream analysis: A survey, *Inf. Fusion* 37 (2017) 132–156.
- [9] N. Gunasekara, H.M. Gomes, B. Pfahringer, A. Bifet, Online hyperparameter optimization for streaming neural networks, in: *Proceedings of the 2022 International Joint Conference on Neural Networks*, 2022, pp. 1–9.
- [10] J. Shan, H. Zhang, W. Liu, Q. Liu, Online active learning ensemble framework for drifted data streams, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (2) (2018) 486–498.
- [11] H. Guo, S. Zhang, W. Wang, Selective ensemble-based online adaptive deep neural networks for streaming data with concept drift, *Neural Netw.* 142 (2021) 437–456.
- [12] D. Sahoo, Q. Pham, J. Lu, S.C. Hoi, Online deep learning: learning deep neural networks on the fly, in: *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, 2018, pp. 2660–2666.
- [13] L. Yuan, H. Li, B. Xia, C. Gao, M. Liu, W. Yuan, X. You, Recent advances in concept drift adaptation methods for deep learning, in: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 2022, pp. 5654–5661.
- [14] Y. Yang, D.-W. Zhou, D.-C. Zhan, H. Xiong, Y. Jiang, Adaptive deep models for incremental learning: Considering capacity scalability and sustainability, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 74–82.
- [15] S. Shiva, M. Hu, P.N. Suganthan, Online learning using deep random vector functional link network, *Eng. Appl. Artif. Intell.* 125 (2023) 106676.
- [16] A. Ashfahani, M. Pratama, Autonomous deep learning: Continual learning approach for dynamic environments, in: *Proceedings of the 2019 SIAM International Conference on Data Mining*, 2019, pp. 666–674.
- [17] M. Pratama, C. Za'in, A. Ashfahani, Y.S. Ong, W. Ding, Automatic construction of multi-layer perceptron network from streaming examples, in: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019, pp. 1171–1180.
- [18] J. Yosinski, J. Clune, Y. Bengio, H. Lipson, How transferable are features in deep neural networks?, in: *Proceedings of the 27th International Conference on Neural Information Processing Systems*, 2014, pp. 3320–3328.
- [19] Z. Li, D. Hoiem, Learning without forgetting, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (12) (2017) 2935–2947.
- [20] F. Bayram, B.S. Ahmed, A. Kassler, From concept drift to model degradation: An overview on performance-aware drift detectors, *Knowl.-Based Syst.* 245 (2022) 108632.
- [21] A.L. Suárez-Cetrulo, D. Quintana, A. Cervantes, A survey on machine learning for recurring concept drifting data streams, *Expert Syst. Appl.* 213 (2022) 118934.
- [22] A. Bifet, R. Gavalda, Learning from time-changing data with adaptive windowing, in: *Proceedings of the 2007 SIAM International Conference on Data Mining*, 2007, pp. 443–448.
- [23] C. Alippi, M. Roveri, Just-in-time adaptive classifiers—Part II: Designing the classifier, *IEEE Trans. Neural Netw.* 19 (12) (2008) 2053–2064.
- [24] C. Alippi, G. Boracchi, M. Roveri, Just-in-time classifiers for recurrent concepts, *IEEE Trans. Neural Netw. Learn. Syst.* 24 (4) (2013) 620–634.
- [25] B. Halstead, Y.S. Koh, P. Riddle, R. Pears, M. Pechenizkiy, A. Bifet, G. Olivares, G. Coulson, Analyzing and repairing concept drift adaptation in data stream classification, *Mach. Learn.* 111 (10) (2022) 3489–3523.
- [26] J. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: *Proceedings of Advances in Artificial Intelligence—SBIA 2004: 17th Brazilian Symposium on Artificial Intelligence*, 2004, pp. 286–295.
- [27] J. Gama, G. Castillo, Learning with local drift detection, in: *Proceedings of the Second International Conference on Advanced Data Mining and Applications*, 2006, pp. 42–55.
- [28] K. Nishida, K. Yamauchi, Detecting concept drift using statistical testing, in: *Proceedings of the International Conference on Discovery Science*, 2007, pp. 264–269.
- [29] M. Baena-Garcia, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, R. Morales-Bueno, Early drift detection method, in: *Proceedings of the Fourth International Workshop on Knowledge Discovery from Data Streams*, 2006, pp. 77–86.
- [30] I. Frias-Blanco, J. del Campo-Ávila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Diaz, Y. Caballero-Mota, Online and non-parametric drift detection methods based on hoeffding's bounds, *IEEE Trans. Knowl. Data Eng.* 27 (3) (2014) 810–823.
- [31] A. Liu, G. Zhang, J. Lu, Fuzzy time windowing for gradual concept drift adaptation, in: *Proceedings of the 2017 IEEE International Conference on Fuzzy Systems*, 2017, pp. 1–6.
- [32] R.S. Barros, D.R. Cabral, P.M. Gonçalves Jr., S.G. Santos, RDDM: Reactive drift detection method, *Expert Syst. Appl.* 90 (2017) 344–355.
- [33] S.-s. Zhang, J.-w. Liu, X. Zuo, Adaptive online incremental learning for evolving data streams, *Appl. Soft Comput.* 105 (2021) 107255.
- [34] S. Xu, J. Wang, Dynamic extreme learning machine for data stream classification, *Neurocomputing* 238 (2017) 433–449.
- [35] T. Dasu, S. Krishnan, S. Venkatasubramanian, K. Yi, An information-theoretic approach to detecting changes in multi-dimensional data streams, in: *Proceedings of the Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006, pp. 1–24.
- [36] X. Song, M. Wu, C. Jermaine, S. Ranka, Statistical change detection for multi-dimensional data, in: *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2007, pp. 667–676.
- [37] N. Lu, J. Lu, G. Zhang, R.L. De Mantaras, A concept drift-tolerant case-base editing technique, *Artificial Intelligence* 230 (2016) 108–133.
- [38] C. Alippi, M. Roveri, Just-in-time adaptive classifiers—Part I: Detecting nonstationary changes, *IEEE Trans. Neural Netw.* 19 (7) (2008) 1145–1153.
- [39] Y. Zhang, G. Chu, P. Li, X. Hu, X. Wu, Three-layer concept drifting detection in text data streams, *Neurocomputing* 260 (2017) 393–403.
- [40] S. Yu, Z. Abraham, Concept drift detection with hierarchical hypothesis testing, in: *Proceedings of the 2017 SIAM International Conference on Data Mining*, 2017, pp. 768–776.
- [41] S. Disabato, M. Roveri, Learning convolutional neural networks in presence of concept drift, in: *Proceedings of the 2019 International Joint Conference on Neural Networks*, 2019, pp. 1–8.
- [42] M. Pratama, A. Ashfahani, Y.S. Ong, S. Ramasamy, E. Lughofer, Autonomous deep learning: Incremental learning of denoising autoencoder for evolving data streams, 2018, *arXiv preprint arXiv:1809.09081*.
- [43] S. Kauschke, D.H. Lehmann, J. Fürnkranz, Patching deep neural networks for nonstationary environments, in: *Proceedings of the 2019 International Joint Conference on Neural Networks*, 2019, pp. 1–8.
- [44] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A.A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al., Overcoming catastrophic forgetting in neural networks, *Proc. Natl. Acad. Sci.* 114 (13) (2017) 3521–3526.
- [45] R. Pascanu, Y. Bengio, Revisiting natural gradient for deep networks, 2013, *arXiv preprint arXiv:1301.3584*.
- [46] T. Sun, Y. Shao, X. Li, P. Liu, H. Yan, X. Qiu, X. Huang, Learning sparse sharing architectures for multiple tasks, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 8936–8943.
- [47] J. Blackard, Covtype, 1998, *UCI Machine Learning Repository*.
- [48] R. Elwell, R. Polikar, Incremental learning of concept drift in nonstationary environments, *IEEE Trans. Neural Netw.* 22 (10) (2011) 1517–1531.
- [49] M. Harries, N.S. Wales, et al., Spline-2 comparative evaluation: Electricity pricing, 1999.
- [50] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [51] H. Xiao, K. Rasul, R. Vollgraf, Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017, *arXiv preprint arXiv:1708.07747*.

- [52] A. Bifet, G. Holmes, R. Kirkby, B. Pfahringer, MOA: Massive online analysis, *J. Mach. Learn. Res.* 11 (2010) 1601–1604.
- [53] M. Lichman, et al., Led display domain, 2013, UCI Machine Learning Repository.
- [54] R. Agrawal, T. Imielinski, A. Swami, Database mining: A performance perspective, *IEEE Trans. Knowl. Data Eng.* 5 (6) (1993) 914–925.
- [55] S. Bakhshi, P. Ghahramanian, H. Bonab, F. Can, A broad ensemble learning system for drifting stream classification, *IEEE Access* 11 (2023) 89315–89330.
- [56] C.P. Chen, Z. Liu, Broad learning system: An effective and efficient incremental learning system without the need for deep architecture, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (1) (2017) 10–24.
- [57] P. Domingos, G. Hulten, Mining high-speed data streams, in: *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2000, pp. 71–80.
- [58] B. Igelnik, Y.-H. Pao, Stochastic choice of basis functions in adaptive function approximation and the functional-link net, *IEEE Trans. Neural Netw.* 6 (6) (1995) 1320–1329.